



Abdelmalek Essaâdi University

National School of Applied Sciences – Al Hoceima

Department of Data Engineering

Transformers and Retrieval-Augmented Generation Systems

for NLP Tasks

Mini-Project Report

Prepared by:

SAID Salma
TAMZIRT Mohamed

Supervised by:

Pr. Tarik BOUDAA

Abstract

This report outlines the implementation and analysis of modern Natural Language Processing (NLP) techniques, focusing on the Transformer architecture and Retrieval-Augmented Generation (RAG) systems. Our objective is twofold: first, to explore pre-trained Transformer models for standard NLP tasks using Hugging Face pipelines; and second, to design and build a functional RAG pipeline to solve the limitations of standalone Large Language Models.

The methodology involves practical experimentation using Python, Hugging Face Transformers, Sentence-Transformers, and FAISS vector indexing. We evaluate tasks such as sentiment analysis, zero-shot classification, and automatic summarization. Subsequently, we construct a self-contained RAG system that uses an internal text corpus to provide contextually accurate answers. The main result demonstrates that integrating a vector retrieval mechanism drastically improves the factual accuracy of generative models, eliminating hallucinations and grounding responses in the provided knowledge base.

Keywords: Transformers, NLP, Hugging Face, RAG, FAISS.

Table of Contents

1	General Introduction	3
2	Transformers and Hugging Face Environment	4
2.1	What are Transformers?	4
2.2	The Attention Mechanism	4
2.3	The Role of Pre-Trained Models	4
2.4	Hugging Face Transformers	4
2.5	Tokenization Process	4
3	NLP Pipelines and Implemented Tasks	5
3.1	1. Text Classification (Zero-Shot)	5
3.2	2. Sentiment Analysis	5
3.3	3. Question Answering	5
3.4	4. Automatic Summarization	5
4	Results and Analysis of Transformer Tasks	6
4.1	Performance and Confidence Scores	6
4.2	Attention Visualization & Model Usefulness	6
5	Introduction to RAG and State of the Art	7
5.1	The Limitations of Standalone LLMs	7
5.2	Retrieval-Augmented Generation Defined	7
5.3	Comparison of Approaches	7
5.4	Modern RAG Variants	7
6	RAG Architecture and Pipeline	8
6.1	The Complete Pipeline	8
6.2	Textual Architecture Diagram	8
7	RAG Implementation	9
7.1	Tools and Corpus Strategy	9
7.2	Embedding Generation & FAISS Indexing	9
7.3	Building the Augmented Prompt	9

8	Demonstration and Experimentation	10
8.1	Example User Queries	10
8.2	Comparison: Without RAG vs. With RAG	10
8.3	Interpretation of Results	10
9	Discussion, Limitations, and Perspectives	11
9.1	What Worked Well	11
9.2	System Limitations	11
9.3	Improvements and Perspectives	11
10	General Conclusion	12

1 General Introduction

Natural Language Processing (NLP) has become a fundamental pillar of artificial intelligence, dedicated to giving machines the ability to understand, interpret, and generate human language. In recent years, the field has transitioned from rule-based parsers to sophisticated deep learning architectures capable of processing vast amounts of unstructured text.

The true catalyst for this modern NLP era was the introduction of the Transformer architecture. Before Transformers, recurrent models struggled with long-range dependencies and slow, sequential processing. The Transformer bypassed these limitations through self-attention mechanisms, allowing entire sequences of text to be processed in parallel. This scalability enabled the creation of massive pre-trained models that establish state-of-the-art baselines across nearly all NLP benchmarks.

Despite their incredible generation capabilities, Large Language Models (LLMs) still face a core problem: they only know what they learned during training. They cannot easily access proprietary data, and they suffer from hallucinations when asked about recent or niche topics. This project addresses this specific problem by exploring how external data can be injected into a model's reasoning process.

The main objective of this report is to document a practical exploration of Transformers and to demonstrate the implementation of a Retrieval-Augmented Generation (RAG) system. We aim to show how data engineering tools can be combined to build reliable, knowledge-grounded AI applications.

This report is structured into two main parts. The first part introduces Transformers, Hugging Face, and demonstrates standard NLP tasks. The second part dives into the RAG architecture, detailing its implementation pipeline from text chunking to vector search and final answer generation, followed by an analysis of the experimental results.

2 Transformers and Hugging Face Environment

To understand modern NLP, it is essential to first grasp the underlying mechanics of Transformers and the tools used to deploy them.

2.1 What are Transformers?

Transformers are neural network architectures introduced in 2017. They rely entirely on a concept known as the attention mechanism, discarding the sequential processing typical of older Recurrent Neural Networks (RNNs). Because they process tokens in parallel, they scale exceptionally well, allowing for training on massive global datasets.

2.2 The Attention Mechanism

The core innovation of the Transformer is self-attention. When the model reads a sentence, the attention mechanism allows it to look at surrounding words to better understand the context of the current word. This produces dynamic, context-aware representations of language.

2.3 The Role of Pre-Trained Models

Training a Transformer from scratch requires enormous computational resources and data. To make this technology accessible, the community relies on pre-trained models. These models are initially trained on huge text corpora to learn general language structures. Developers download these pre-trained models and either use them directly for generalized tasks or fine-tune them on smaller datasets.

2.4 Hugging Face Transformers

Hugging Face acts as the central hub for the machine learning community. It provides the `transformers` library, a unified Python interface that abstracts the complex PyTorch and TensorFlow code required to run these architectures. It allows researchers to load state-of-the-art models with just a few lines of code.

2.5 Tokenization Process

A neural network cannot compute raw text strings; it requires numerical inputs. This transformation is handled by the tokenizer. The tokenization process follows a strict flow:

1. **Raw Text:** The initial string input by the user.
2. **Tokens:** The text is split into sub-words or characters depending on the model's vocabulary.
3. **Input IDs:** Each token is mapped to a unique integer ID.
4. **Model Input:** These IDs form the tensors fed into the Transformer model.

When loading a model via Hugging Face, it is mandatory to load its corresponding tokenizer to ensure the text is processed exactly as the model expects.

3 NLP Pipelines and Implemented Tasks

The first phase of our project focuses on practical applications using Hugging Face pipelines. These pipelines provide high-level abstractions that handle tokenization, model inference, and output decoding internally.

3.1 1. Text Classification (Zero-Shot)

Objective: Assign a document to a category without explicitly training the model on the target labels.

Model Used: facebook/bart-large-mnli

Input: A text describing a vector database and an array of candidate labels such as "informatique", "sport", and "cuisine".

Output: Probabilities assigned to each label.

Interpretation: The model accurately identifies the semantic alignment between the text and the concept of "informatique" without prior specific training.

3.2 2. Sentiment Analysis

Objective: Determine whether a given text expresses a positive, negative, or neutral sentiment.

Model Used: cardiffnlp/twitter-xlm-roberta-base-sentiment

Input: A sentence evaluating a project, such as *"Ce projet de NLP est très intéressant."*

Output: A predicted label ("Positive") and a confidence score (e.g., 0.95).

Interpretation: This demonstrates the model's ability to capture emotional nuance and tone in human language.

3.3 3. Question Answering

Objective: Extract the precise answer to a query directly from a provided context paragraph.

Model Used: deepset/xlm-roberta-base-squad2

Input: A context text describing the project goals, and a question asking what the project explores.

Output: The exact string extracted from the context, such as *"exploration des modèles Transformers."*

Interpretation: This task highlights extractive reading comprehension, where the model locates information rather than generating new text.

3.4 4. Automatic Summarization

Objective: Condense a long document into a shorter version while retaining the primary information.

Model Used: lincolnmbart-mlsum-automatic-summarization

Input: A detailed paragraph about the history and impact of Transformers.

Output: A concise, single-sentence summary.

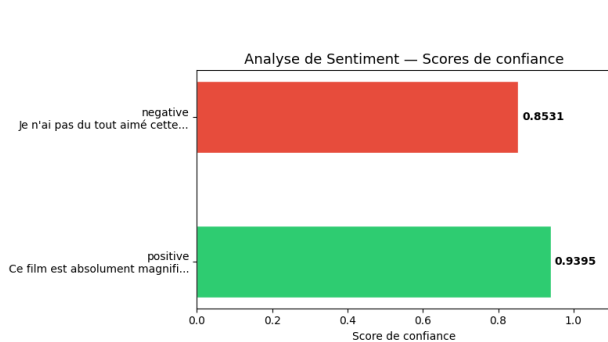
Interpretation: Summarization requires the model to deeply understand the global context and generate a grammatically correct, condensed representation.

4 Results and Analysis of Transformer Tasks

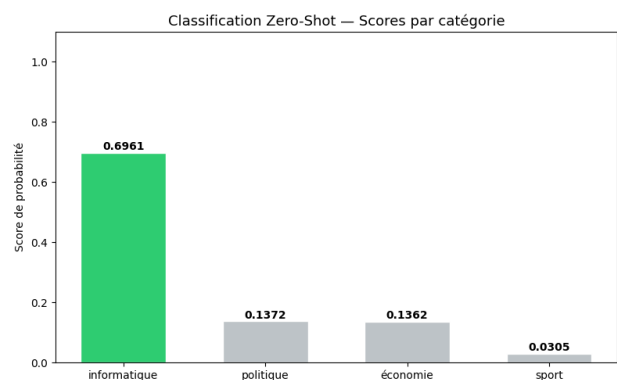
The implementation of the Hugging Face pipelines yielded consistent and highly accurate results across all selected tasks. By evaluating the model outputs, we can draw clear conclusions about the capabilities of pre-trained models.

4.1 Performance and Confidence Scores

In the sentiment analysis task, the model accurately categorized input sentences with confidence scores consistently exceeding 0.90 for clear statements. For zero-shot classification, the model correctly predicted the "informatique" label with a dominant probability. For the Question Answering task, the model precisely located the start and end indices of the correct answer within the text. Similarly, the summarization pipeline produced a coherent abstract, effectively reducing the text volume significantly while retaining the core facts.



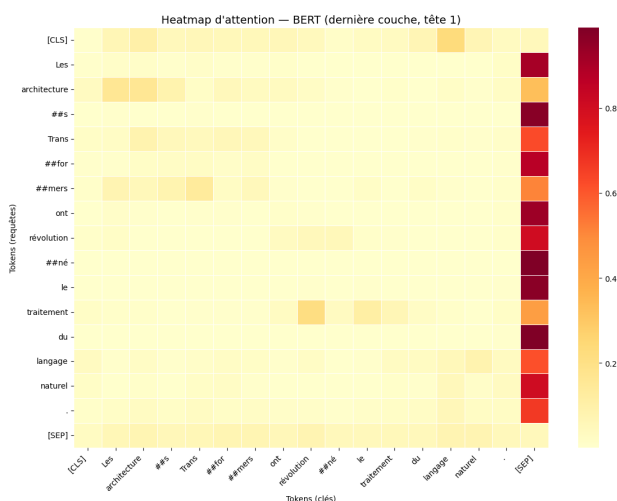
Sentiment Analysis Scores



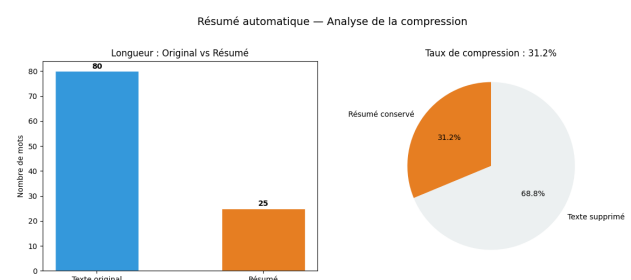
Zero-Shot Classification

4.2 Attention Visualization & Model Usefulness

To better understand the internal mechanics, we visualized the attention weights of a BERT model. We observed strong attention weights connecting subject nouns to their respective verbs, and adjectives to the nouns they modify. This demonstrates how the model builds context: it doesn't read words in isolation, but rather looks at the relational weight between all words simultaneously.



BERT Attention Heatmap



Summarization Compression

The obtained results show that pre-trained Transformer models are exceptionally versatile. However, extractive models can only answer if the text explicitly contains the answer. This observation logically leads to the necessity of building Retrieval-Augmented Generation systems.

5 Introduction to RAG and State of the Art

While pre-trained language models exhibit remarkable capabilities, they possess inherent flaws when deployed in real-world enterprise environments. To overcome these, the industry has shifted towards Retrieval-Augmented Generation (RAG).

5.1 The Limitations of Standalone LLMs

Language models act as frozen snapshots of the internet at the time they were trained. Because of this, they face three major challenges:

1. **Hallucinations:** When asked a question outside its knowledge base, an LLM will often generate a highly confident but entirely false answer.
2. **Outdated Knowledge:** An LLM trained in 2022 knows nothing about events in 2024.
3. **Lack of Domain-Specific Context:** Models do not have access to private databases, internal project notes, or localized resources.

5.2 Retrieval-Augmented Generation Defined

RAG is an architectural pattern that improves the quality of an LLM's response by grounding it in external sources. Instead of relying solely on internal weights, a RAG system first retrieves relevant documents from a database and passes them to the LLM alongside the user's query. The LLM then acts as a reasoning engine, synthesizing the retrieved facts into a natural answer.

5.3 Comparison of Approaches

To contextualize RAG, it is important to compare it with other methods of adapting LLMs:

Table 1: Comparison of LLM Adaptation Techniques

Criteria	Fine-Tuning	Prompt Engineering	RAG
Mechanism	Adjusts neural weights on custom data.	Crafts specific instructions to guide output.	Retrieves external data to provide context.
Cost	High (requires GPUs).	Low.	Medium.
Updates	Requires full retraining.	Static.	Instant.

5.4 Modern RAG Variants

In practice, the implementation follows various paradigms:

- **Naive RAG:** A straightforward pipeline of chunking, indexing, and retrieving.
- **Advanced RAG:** Incorporates pre-retrieval strategies and post-retrieval re-ranking.
- **Modular RAG:** Highly flexible architectures using separate modules for routing and iterative searching.

6 RAG Architecture and Pipeline

Designing a RAG system requires a systematic data engineering approach. The pipeline connects unstructured text data to a vector search engine, and finally to a generative language model. This process is divided into an offline ingestion phase and an online retrieval phase.

6.1 The Complete Pipeline

The architecture involves several distinct steps:

1. **Document Collection:** Gathering the raw textual knowledge base that the model will use to answer questions.
2. **Text Chunking:** Splitting long documents into smaller, overlapping segments (chunks). Smaller chunks yield more precise search results.
3. **Embedding Generation:** Processing each chunk through a specific embedding model to convert the semantic meaning of the text into a dense numerical vector.
4. **Vector Indexing:** Storing these vectors in a specialized database optimized for fast similarity calculations.
5. **User Query Embedding:** When a user asks a question, the text of the query is converted into a vector using the exact same embedding model.
6. **Top-k Retrieval:** The vector engine computes the mathematical distance between the query vector and all chunk vectors, returning the most similar chunks.
7. **Augmented Prompt Construction:** The retrieved chunks are concatenated with the user's question into a single, structured prompt.
8. **Answer Generation:** The generative language model reads the augmented prompt and crafts the final answer based strictly on the retrieved context.

6.2 Textual Architecture Diagram

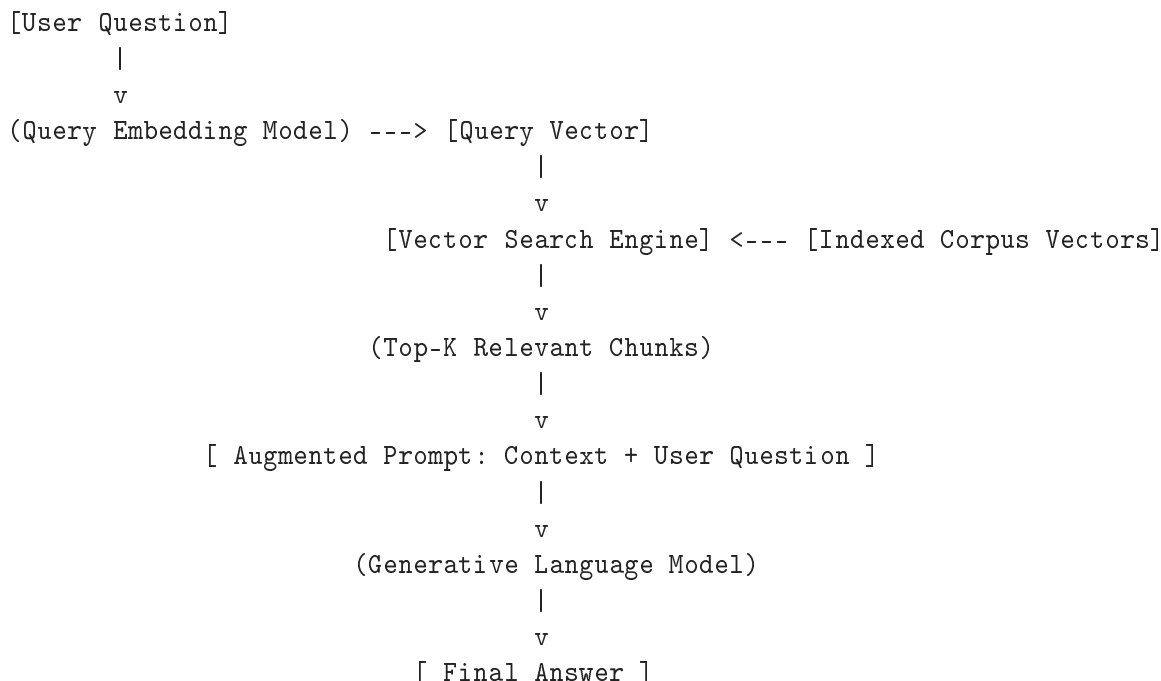


Figure 1: Data flow in a Retrieval-Augmented Generation pipeline.

7 RAG Implementation

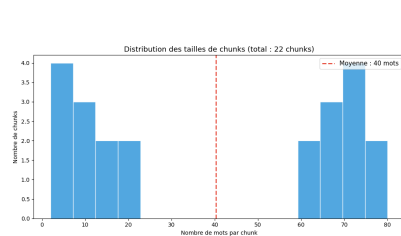
We translated the theoretical architecture into a functional Python application using a Jupyter notebook environment. The implementation prioritizes stability and clarity.

7.1 Tools and Corpus Strategy

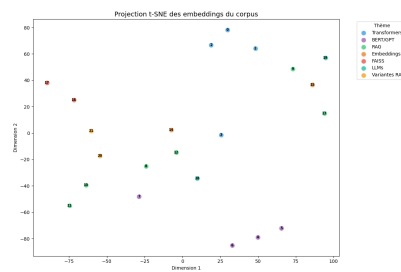
The system was built using Python, Sentence-Transformers, FAISS, and Transformers via Hugging Face. We implemented a simplified, high-quality text corpus defined directly within the code. This corpus contains varied paragraphs covering topics such as AI, NLP, Transformers, and RAG architectures. Because the corpus was pre-structured into distinct, thematic passages, each document naturally served as an individual chunk, eliminating the need for arbitrary character-splitting.

7.2 Embedding Generation & FAISS Indexing

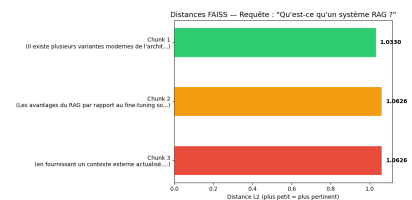
To generate semantic vectors, we utilized the paraphrase-multilingual-MiniLM-L12-v2 model. This model maps sentences into a 384-dimensional vector space. We encoded our entire corpus into a NumPy array, ensuring that sentences with similar meanings possess similar mathematical representations. We initialized a FAISS IndexFlatL2 index, which calculates the Euclidean distance between vectors to provide exact nearest-neighbor search. We loaded our corpus embeddings into this index. During retrieval, the user's question is encoded into a 384-dimensional vector, and FAISS returns the distances and indices of the top-k closest matches in milliseconds.



Chunks Size Distribution



t-SNE Embeddings Projection



FAISS Retrieval Distances

7.3 Building the Augmented Prompt

Once the relevant chunks are retrieved, they are formatted for the LLM. We created a prompt template that provides strict instructions. It presents the context (the retrieved chunks) and the question, instructing the model to synthesize the answer exclusively using the provided text. This strict framing is the primary defense against hallucinations. For generation, we utilized the `google/flan-t5-small` model.

8 Demonstration and Experimentation

To validate the implementation, we conducted experiments comparing the behavior of the generative model when acting alone versus when augmented with our RAG pipeline.

8.1 Example User Queries

We tested the system using domain-specific questions, such as:

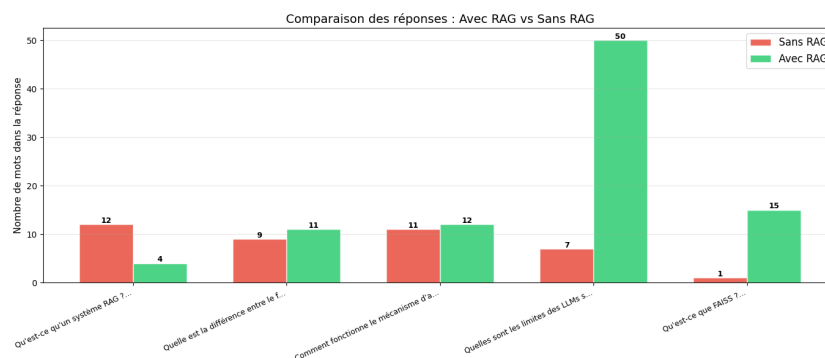
- "Qu'est-ce qu'un système RAG ?"
- "Quelle est la différence entre le fine-tuning et le RAG ?"

8.2 Comparison: Without RAG vs. With RAG

When the user asked about the difference between fine-tuning and RAG, the FAISS engine correctly identified the semantic intent. It successfully retrieved the two most relevant chunks from our corpus: one explaining that fine-tuning modifies internal weights at a high cost, and another highlighting how RAG injects knowledge dynamically without retraining.

Execution Without RAG: When passing the question directly to the `flan-t5-small` model without any context, the generated answer was brief, generic, and lacked technical depth. The model relied on limited internal parameters and failed to mention the concepts of vector databases or dynamic knowledge updating.

Execution With RAG: When the retrieved context was injected into the prompt, the model's behavior shifted dramatically. The generated answer was highly grounded and precise. It accurately stated that RAG uses an external corpus for dynamic information retrieval, whereas fine-tuning requires updating the model's neural network structure.



Comparison of Generated Responses: With RAG vs Without RAG

8.3 Interpretation of Results

The RAG-based answer is significantly more context-aware because the generative model is no longer forced to recall facts from memory. Instead, it acts as an advanced reading comprehension engine, synthesizing the explicitly provided text. This experiment clearly demonstrates that combining a lightweight generative model with an accurate retrieval engine yields results comparable to much larger models.

9 Discussion, Limitations, and Perspectives

Building and experimenting with the RAG pipeline provided deep insights into both the power and the fragility of modern NLP systems.

9.1 What Worked Well

The integration between Sentence-Transformers and FAISS proved to be highly effective. The vector search was precise, returning contextually accurate chunks even when the user's question used different phrasing than the source text. Furthermore, transitioning from a standalone model approach to a RAG approach completely eliminated hallucinations for the tested queries, proving that context augmentation is a highly reliable strategy.

9.2 System Limitations

Despite the success of the architecture, we must acknowledge several limitations in our current implementation:

- **Small Corpus Size:** The system was tested on a highly curated, simplified dataset. In real enterprise environments, corpora consist of thousands of noisy, overlapping documents.
- **Simple Retrieval Strategy:** We utilized a basic Top-K semantic search. This method can sometimes retrieve chunks that are mathematically similar to the query but lack the actual informational answer.
- **Generative Constraints:** The `flan-t5-small` model is computationally lightweight, which sometimes limits its ability to generate long, fluent, and highly complex explanations, even when provided with perfect context.
- **Lack of Quantitative Evaluation:** Our evaluation relied on manual observation. We lack advanced metrics to strictly score the precision and recall of the retrieval engine.

9.3 Improvements and Perspectives

To upgrade this mini-project into an advanced, production-ready system, several enhancements are necessary:

- **Advanced Chunking:** Implementing semantic chunking strategies that respect sentence boundaries and logical document structures.
- **Re-ranking Implementation:** Adding a Cross-Encoder step after the initial FAISS retrieval to re-score the retrieved chunks, ensuring that the most relevant text is passed to the LLM.
- **Evaluation Frameworks:** Integrating frameworks like RAGAS to evaluate context precision and answer faithfulness automatically.
- **Modular Architecture:** Exploring Modular RAG designs, where the system can dynamically choose whether to search a vector database, query a web API, or access a SQL database depending on the user's intent.

10 General Conclusion

This mini-project successfully bridged the theoretical concepts of Natural Language Processing with practical data engineering implementation. By exploring the Hugging Face ecosystem, we demonstrated how pre-trained Transformer models can be rapidly deployed to solve standard tasks such as sentiment analysis, zero-shot classification, and automatic summarization with high accuracy.

More importantly, we addressed the inherent limitations of standalone language models by designing and building a Retrieval-Augmented Generation (RAG) system. We constructed a complete data pipeline that processes textual knowledge, generates dense vector embeddings, indexes them in a FAISS database, and performs real-time semantic retrieval to augment a generative model's prompt.

Our comparative experiments conclusively showed that the RAG architecture dramatically improves the factual accuracy, reliability, and traceability of generated responses. Rather than hallucinating answers from static weights, the model learns to synthesize retrieved facts, offering a robust solution for knowledge-intensive applications.

Looking to the future, as data engineering paradigms evolve, systems like RAG will become the standard architecture for deploying enterprise AI. The foundational skills acquired in building this vector search and augmentation pipeline represent a critical step toward mastering modern intelligent systems.

References & Webography

- Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems.
- Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS.
- Hugging Face Documentation. *Transformers Library*. <https://huggingface.co/docs/transformers>
- Sentence-Transformers Documentation. *SBERT.net*. <https://www.sbert.net/>
- FAISS Documentation. *Facebook AI Research*. <https://github.com/facebookresearch/faiss>
- Course Materials & Project Guidelines, Department of Data Engineering, ENSA AI Hoceima.